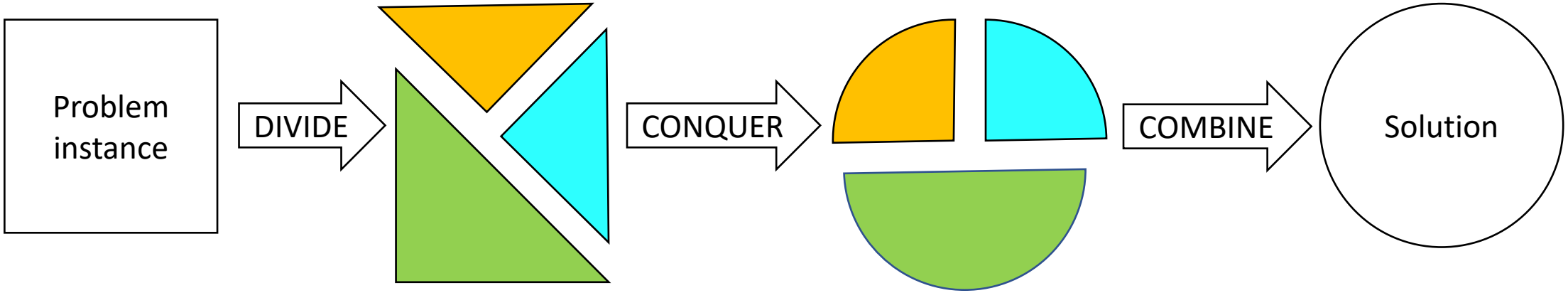
Thinking in parallel

How to design algorithms for multi-processor systems

Basic techniques

Basic techniques: Divide and Conquer

- It consists of three steps
  1. **DIVIDE**: partition the input instance into independent sub-instances, usually of equal size
  2. **CONQUER**: solve the sub-instances, hopefully concurrently
     - The sub-instances are solved recursively (classical Divide And Conquer)
     - The sub-instances are solved directly (Divide and Conquer is also called **PARTITIONING**)
  3. **COMBINE**: combine the solutions of the sub-instances to obtain a solution of the instance



Merging problem

MERGING
Input: Two sorted arrays A and B of n elements each.
Output: A sorted array C obtained by merging A and B.

EXAMPLE:

A	1	6	7	9
B	2	3	5	8

C	1	2	3	5	6	7	8	9

AN $O(n)$-TIME SEQUENTIAL ALGORITHM
ALGORITHM Merge(A,B)
1. **if** length(A) = 0 **then** return B; **if** length(B) = 0 **then return** A
2. **if** A[1] $\leq$ B[1] **then** C $\leftarrow$ A[1] concatenated with Merge(A[2..length(A)], B)
3. **else** C $\leftarrow$ B[1] concatenated with Merge(A, B[2..length(B])
4. **return** C

it is asymptotically optimal, …
… but it is not parallelizable

The definition of rank of an element

USEFUL ASSUMPTION: W.l.o.g., all elements appearing in A and B are distinct.

If this is not the case, then we can model any element x appearing in the array $Y \in \{A, B\}$ at position i with a triple (x, Y, i) and we define a total ordering $\prec$ of the triples as follows:

$(x, Y, i) \prec (x', Y', i')$ iff $(x < x')$ or $(x = x'$ and $Y = A$ and $Y' = B)$ or $(x = x'$ and $Y = Y'$ and $i < i')$

DEFINITION: Let Y be an array of t elements. The *rank* of x in Y is

$$rank(x{:}\,Y) = |\{y \in Y : y \leq x\}|.$$

For an array X of s elements,

$$rank(X{:}\,Y) = (r_1, r_2, \dots, r_s), \text{ where } r_i = rank(X[i]{:}\,Y).$$

EXAMPLE:

Y	2	5	6	8

X	3	4	7

$rank(X{:}\,Y)$	1	1	3

A first attempt using partitioning

OBSERVATION: For each $C[i]$, we have $i = rank(C[i]: A \cup B)$.
We compute $rank(x: A \cup B)$ for every $x \in A \cup B$.
Moreover, $rank(x: A \cup B) = rank(x: A) + rank(x: B)$.
For every $x \in A$ (resp., $x \in B$), we already know $rank(x: A)$ (resp., $rank(x: B)$).
For every $x \in A$ (resp., $x \in B$), we compute $rank(x: B)$ (resp., $rank(x: A)$).

OUR FIRST (NON-OPTIMAL) SOLUTION:

- **PARTITIONING**: compute $rank(A: B)$ and $rank(B: A)$.
- **COMBINE**: compute C using $rank(A: B)$ and $rank(B: A)$.

EXAMPLE:

index	1	2	3	4
A	1	6	7	9

index	1	2	3	4
B	2	3	5	8

index	1	2	3	4	5	6	7	8
C	1	2	3	5	6	7	8	9

$rank(5: A \cup B) = rank(5: A) + rank(5: B) = 1 + 3 = 4$

The pseudocode

```
ALGORITHM Merge          /* All elements are distinct */
1.   for 1 ≤ i ≤ n pardo     /* PARTITIONING */
2.       RANK_A[i] ← compute rank(A[i]:B) via binary search
3.       RANK_B[i] ← compute rank(B[i]:A) via binary search
4.   for 1 ≤ i ≤ n pardo     /* COMBINE */
5.       C[i + RANK_A[i]] ← A[i]; C[i + RANK_B[i]] ← B[i]
```

- Time $T(n) = O(\log n)$
- Work $W(n) = O(n \log n)$ (not work optimal)
- CREW PRAM algorithm
  - No concurrent writes on same variable
  - Concurrent reads on same variable in binary search

Towards a work-optimal algorithm

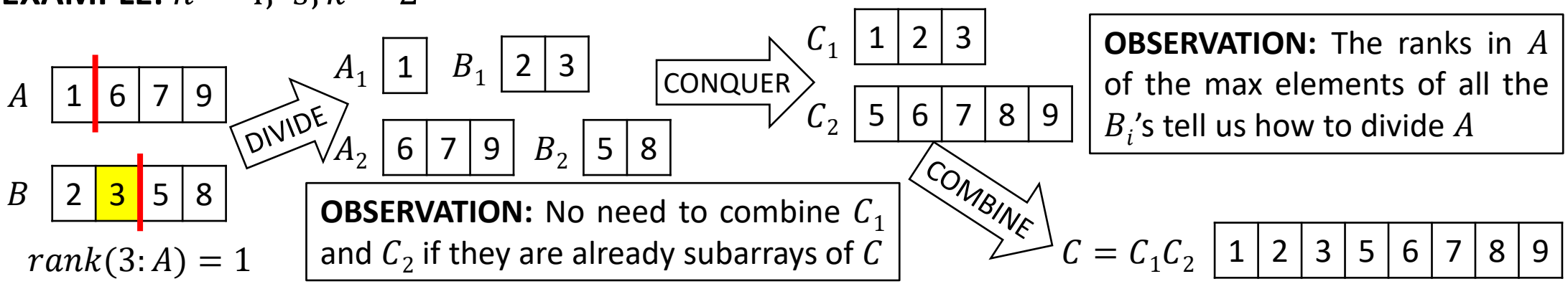
- To simplify, we assume that $s = \log_2 n$ is integral and $k = \frac{n}{\log_2 n}$ is integral (example $n = 2^2$)

TOWARDS OUR OPTIMAL SOLUTION

- Divide A into k blocks $A_1, A_2, \dots, A_k$ and B into k blocks $B_1, B_2, \dots, B_k$ such that
  - each B_i has size s
  - each B_i needs to be merged only with A_i, i.e., $\max_{x \in A_{i-1}} x < \min_{y \in B_i} y$ and $\max_{x \in B_i} x < \min_{y \in A_{i+1}} y$

OBSERVATION: If C_i is the merging of A_i and B_i, then the sequence $C_1, C_2, \dots, C_k$ is sorted

EXAMPLE: $n = 4,\ s, k = 2$



OBSERVATION: No need to combine C_1 and C_2 if they are already subarrays of C

OBSERVATION: The ranks in A of the max elements of all the B_i's tell us how to divide A

The pseudocode for partitioning

ALGORITHM Generate_Subproblems /* DIVIDE */

/* All elements are distinct. A has O(n) elements, B has m=O(n) elements, s is the size of a block */

/* Assume k=m/s is integral */

1. J[0] ← 0; J[k] ← n
2. **for** $1 \le i \le k-1$ **pardo**
3. J[i] ← compute rank(B[i·s]:A) via binary search

Time $O(\log n)$; Work $O\left(\frac{n \cdot \log n}{s}\right) = O(n)$

4. **for** $1 \le i \le k$ **pardo**
5. B_i is B[1+(i –1)·s .. i·s]; A_i is A[J[i-1]+1.. J[i]]

Time $O(1)$ and Work $O(n/s)$

/* Step 5 is conceptual (no need to generate A_i and B_i)*/

/* Some A_i's might be empty (when J[i-1]=J[i])*/

/* We only need to merge A_i and B_i in C_i */

/* C_i =C[J[i-1] +1+(i – 1)·s .. J[i]+i·s] */

- Time $T(n) = O(\log n)$
- Work $W(n) = O\left(\frac{n \cdot \log n}{s}\right)$
- CREW PRAM algorithm
  - Concurrent reads on same variable in binary search

Correctness of the partitioning algorithm

LEMMA: Let C be the sorted array obtained by merging the two sorted arrays A and B, of $O(n)$ elements each. Let $s = \log_2 n$. Then, algorithm Generate_Subproblems partitions A and B into k pairs of subsequences (A_i, B_i) such that (i) $|B_i| = O(\log n)$ and , (ii) $C = C_1 C_2 \dots C_k$, where C_i is the sorted array obtained by merging A_i with B_i. Moreover, the algorithm runs on a CREW PRAM in time $O(\log n)$ and work $O(n)$.

PROOF (only correctness): (i) holds by construction (step 5).

(ii): Let m be the size of B. Let $1 \leq i \leq \frac{m}{s} - 1$.

- The smallest elements of A_{i+1} and B_{i+1} are $A[J[i] + 1]$ and $B[1 + i \cdot s]$ (step 5).
- The largest elements of A_i and B_i are $A[J[i]]$ and $B[i \cdot s]$.
- Since $rank(B[i \cdot s]: A) = J[i]$ (step 3), $A[J[i]] < B[i \cdot s] < A[J[i] + 1]$.
- This implies $B[1 + i \cdot s] > B[i \cdot s] > A[J[i]]$ and $A[J[i] + 1] > B[i \cdot s]$.
- Therefore, each element of A_{i+1} and B_{i+1} is larger than each of the elements in A_i and B_i.

A worst-case instance

- W.l.o.g., we assume that $\log_2 n$ is integral and $k = \frac{n}{\log_2 n}$ is integral (example $n = 2^2$)
- **ISSUE:** Some subarrays of A might still be too large (e. g., $\Omega(n)$ size)

$\Rightarrow$ Computing the ranks of all elements of a $\Theta(n)$-size A_i w.r.t. B_i takes $\Omega(n \log\log n)$ work

EXAMPLE:

	A_1	A_2	A_3						A_4							
A	4	9	14	16	17	18	19	21	22	23	24	26	27	29	30	31

A_4 is too large!

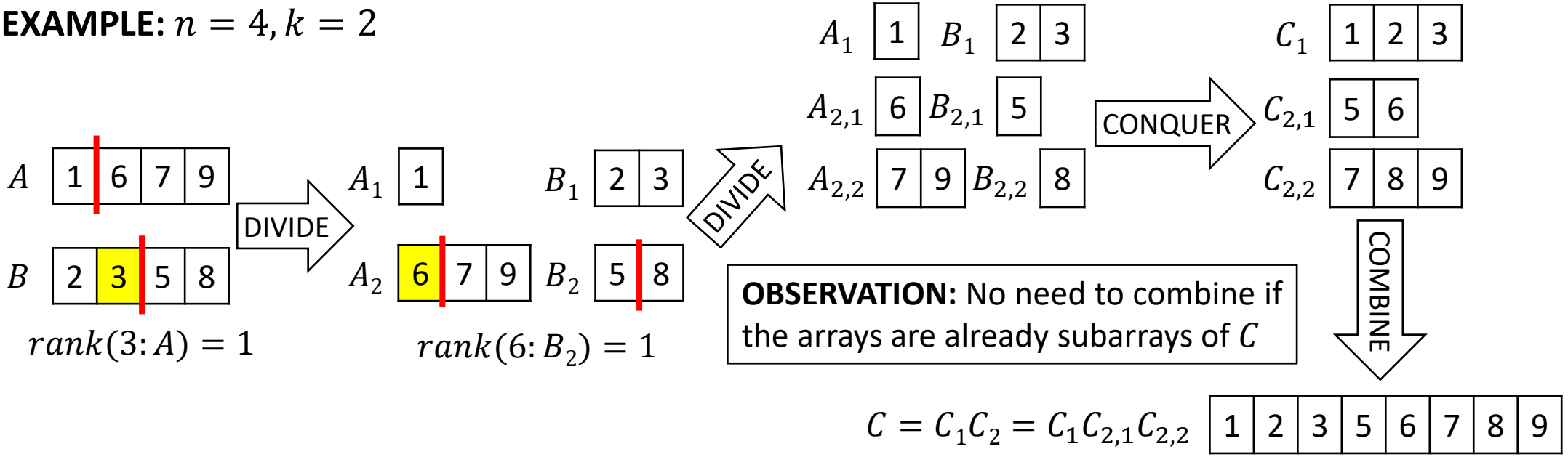
		B_1				B_2				B_3				B_4		
B	1	2	3	5	6	7	8	10	11	12	13	15	20	25	28	32

$rank(5{:}A) = 1 \quad rank(10{:}A) = 2 \quad rank(15{:}A) = 3$

How to fix the worst case

- **ISSUE:** Some subarrays of A might still be too large (e. g., $\Omega(n)$ size)
- **SOLUTION**: Use Generate_Subproblems with parameter $s = \log_2 n$ for every pair (A_i, B_i) such that A_i has a size $> \log_2 n$. In the pseudocode, A_i takes the role of B and B_i takes the role of A.

EXAMPLE: $n = 4, k = 2$



OBSERVATION: No need to combine if the arrays are already subarrays of C

The pseudocode of the work-optimal algorithm

ALGORITHM Optimal_Merge /* All elements are distinct. $\log_2$ n and n/$\log_2$ n are both integral */

/* DIVIDE */

1. Call Generate_Subproblems for the pair (A,B) with s=$\log_2$n Time $O(\log n)$ and work $O(n)$
2. **for** $1 \le i \le n/\log_2 n$ **pardo** Time $O(\log\log n)$ and work $O(n)$
3. Call Generate_Subproblems for the pair (B_i,A_i) with s=$\log_2$n /* it does nothing if $|A_i|<\log_2 n$ */
4. Let (A_1,B_1),…, (A_h,B_h) be the h=O(n/log n) pairs /* $|A_j|,|B_j|$=O(log n) */
5. **for** $1 \le j \le h$ **pardo** /* CONQUER AND COMBINE */
6. Merge A_j and B_j using the <u>sequential</u> algorithm

Time $O(\log n)$ and work $O(h \cdot \log n) = O(n)$

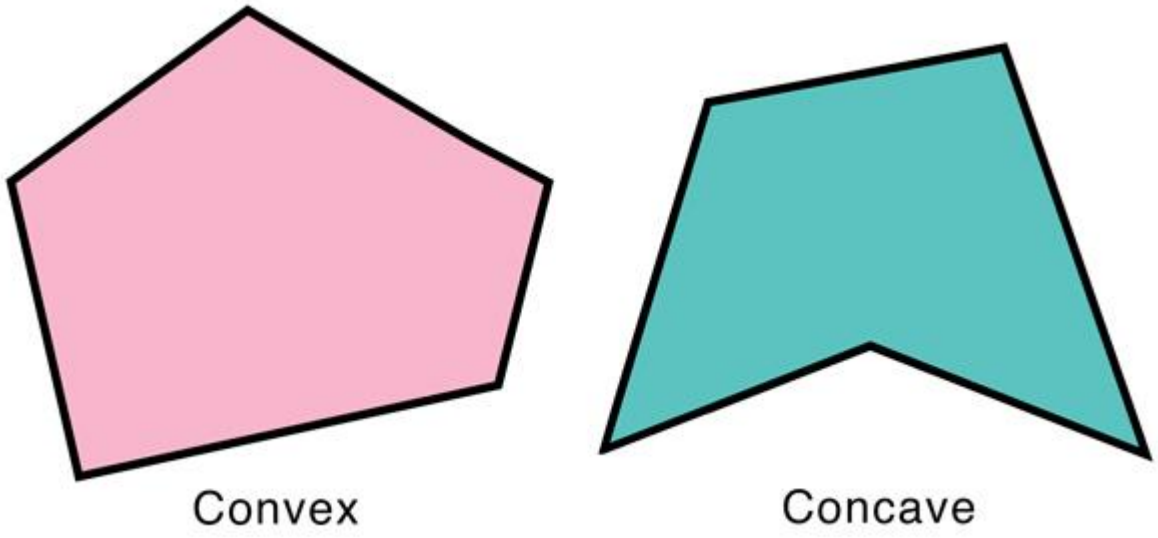
- Time $T(n) = O(\log n)$
- Work $W(n) = O(n)$ (work optimal)
  - The sequential algorithm runs in time $O(|A_j| + |B_j|) = O(\log n)$ for each j
- CREW PRAM algorithm
  - No concurrent write
  - Concurrent reads in Generate_Subproblems

The work-optimal algorithm for merging

THEOREM: Given two sorted arrays A and B with n elements each, we can merge them into a sorted array C on a CREW PRAM in time $O(\log n)$ and work $O(n)$.

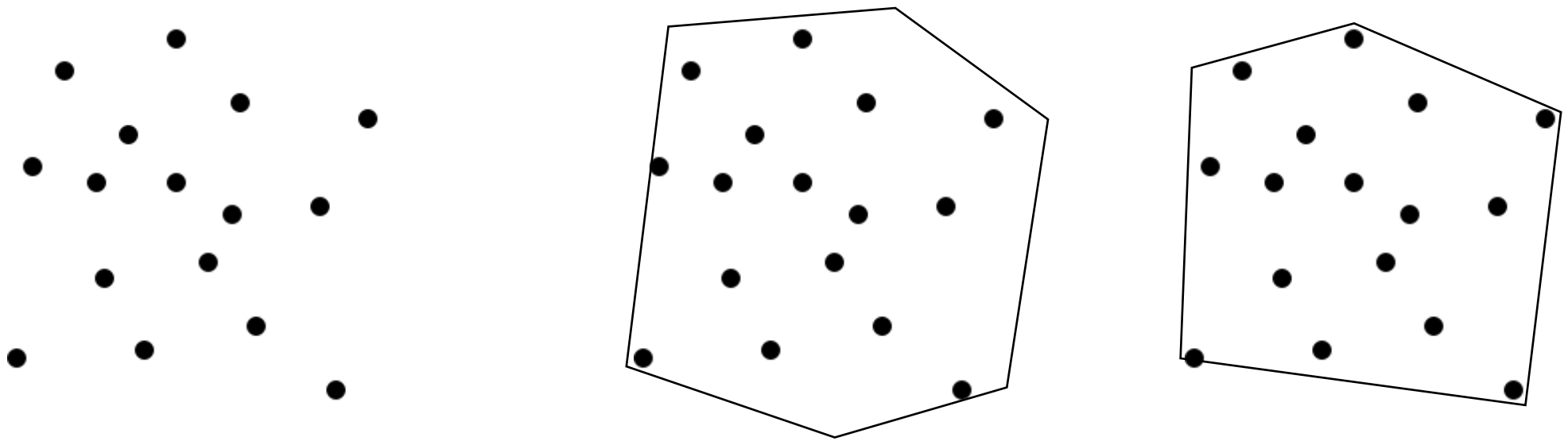
Divide and Conquer: The convex-hull problem

DEFINITION: A polygon P is *convex* if, given any two points p and q of P, all the points along the segment from p to q are contained in P.



The convex-hull problem

CONVEX HULL
Input: A set $S = \{p_1, p_2, \dots, p_n\}$ of n points in the plane, where $p_i = (x_i, y_i)$.
Output: The *convex hull* $CH(S)$ of S, i.e., the smallest convex polygon containing S

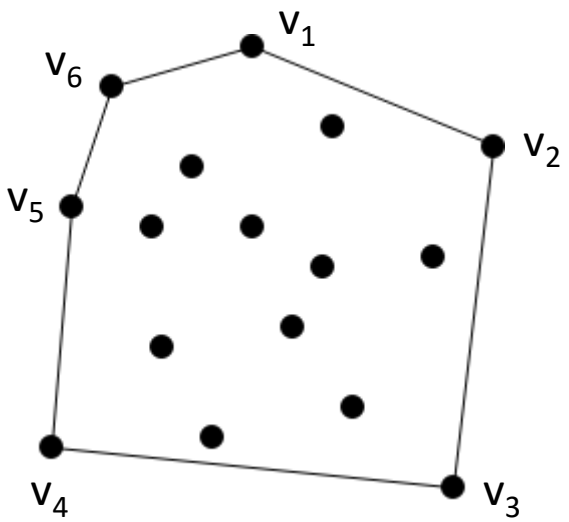


Assumption 1: For every $(x, y), (x', y') \in S$, we have $x \neq x'$ and $y \neq y'$.

A useful lemma

CONVEX HULL
Input: A set $S = \{p_1, p_2, \dots, p_n\}$ of n points in the plane, where $p_i = (x_i, y_i)$.
Output: The *convex hull* $CH(S)$ of S, i.e., the smallest convex polygon containing S

LEMMA (without proof): $CH(S)$ is a polygon whose vertices are all contained in S.



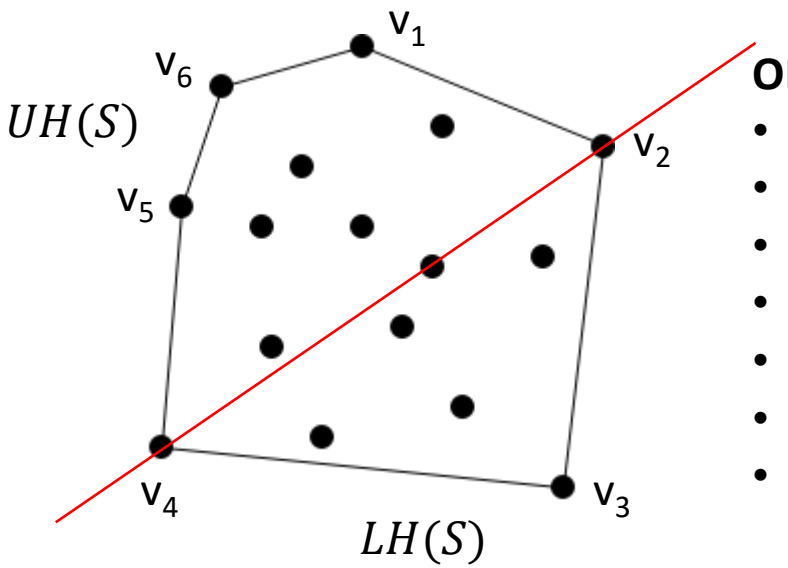
OBSERVATIONS:
- $CH(S)$ can be described by the circular list of its vertices, in clockwise order
- By the Lemma, the description of $CH(S)$ is a list of some points in S

Assumption 2: S does not contain three or more points that are aligned.

Subdivision into upper hull and lower hull

CONVEX HULL
Input: A set $S = \{p_1, p_2, \dots, p_n\}$ of n points in the plane, where $p_i = (x_i, y_i)$.
Output: The *convex hull* $CH(S)$ of S, i.e., the smallest convex polygon containing S

LEMMA (without proof): $CH(S)$ is a polygon whose vertices are all contained in S.



OBSERVATIONS:

- $CH(S)$ can be described by the circular list of its vertices, in clockwise order
- By the Lemma, the description of $CH(S)$ is a list of some points in S
- The points in S of min/max x-coordinate, say p_{min} and p_{max} are in $CH(S)$
- $UH(S)$ or *upper hull* of S: points in S from p_{min} to p_{max} in clockwise order
- $LH(S)$ or *lower hull* of S: points in S from p_{max} to p_{min} in clockwise order
- We can compute $CH(S)$ by concatenating $UH(S)$ with $LH(S)$
- We show how to compute $UH(S)$ ($LH(S)$ can be computed similarly)

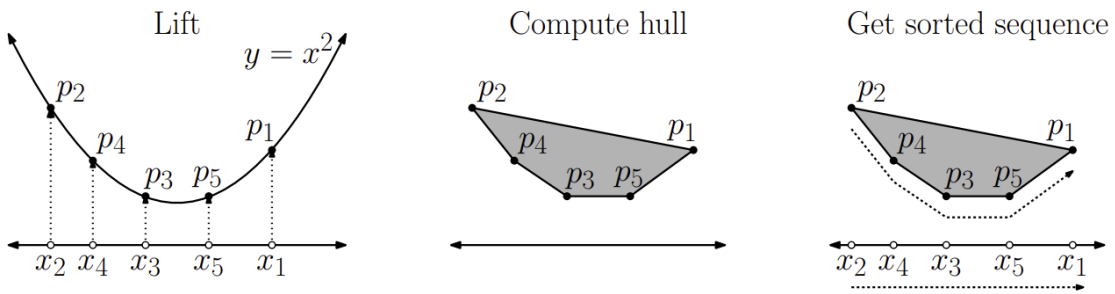
Sequential algorithm and lower bound

- There is an $O(n \log n)$ time sequential algorithm that computes the convex hull of a set S of n points

LEMMA: Any comparison-based sequential algorithm requires $\Omega(n \log n)$ time to compute the convex hull of a set S of n points.

SKETCH OF PROOF: Reduction from sorting an array X on n elements $x_1, \dots, x_n$, a problem that requires $\Omega(n \log n)$ in comparison-based models of sequential algorithms.
$S = \{p_i = (x_i, x_i^2)\colon 1 \le i \le n\}$ are subset of points of the parabola $y = x^2$

- All points are vertices in the convex hull



The counter-clockwise order of points along the lower hull gives the elements of X sorted

How to compute the upper hull

LEMMA (without proof): We can sort an array of n elements on an EREW PRAM in time $O(\log n)$ and work $O(n \log n)$.

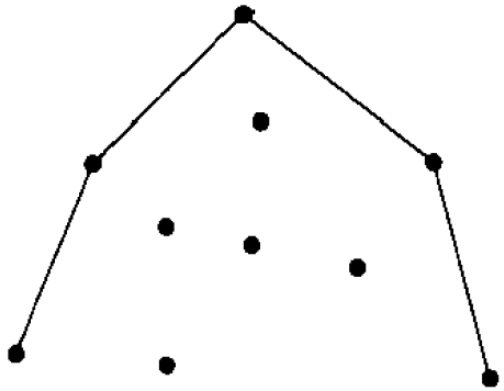
IDEA: Assume that $n = 2^k$, for some integer k

- PREPROCESSING: Sort all the points in S w.r.t. their x-coordinates. Renumber the points in S in such a way that $p_1, \dots, p_n$ is the sequence of sorted points
- DIVIDE + CONQUER
  - Concurrently compute the upper hulls $UH(S_1)$ of $S_1 = \{p_1, \dots, p_{n/2}\}$ and $UH(S_2)$ of $S_2 = \{p_{\frac{n}{2}+1}, \dots, p_n\}$ recursively
- COMBINE: Compute $UH(S)$ from $UH(S_1)$ and $UH(S_2)$

How to combine the two upper hulls

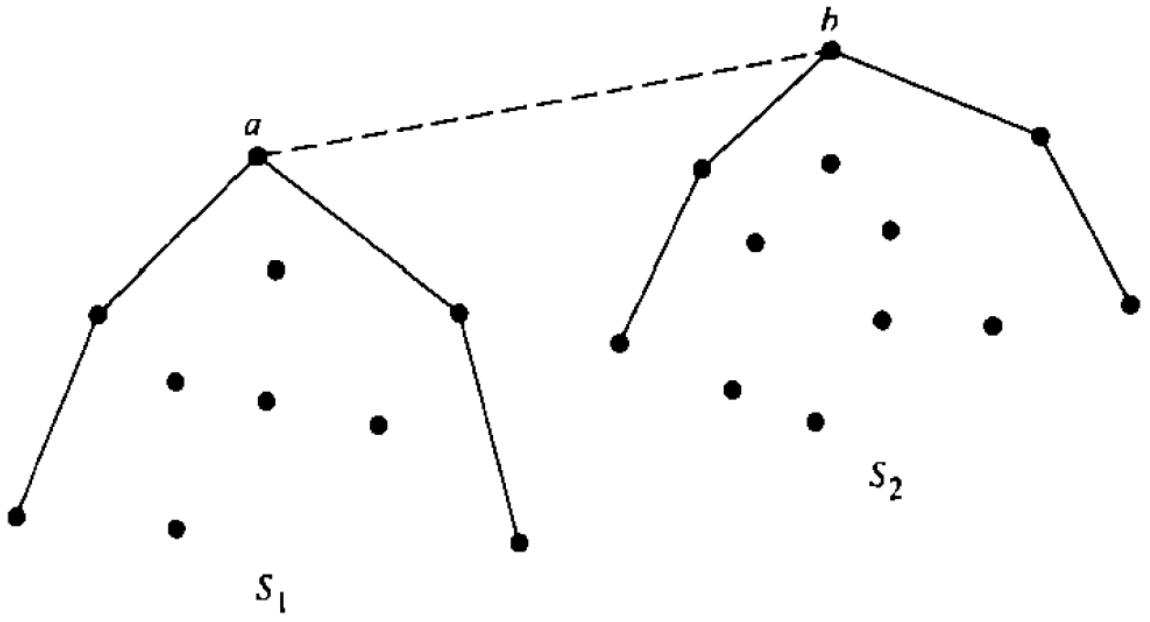
LEMMA: The upper hull $UH(S)$ is a list of points $q_1, \dots, q_h$ for which there exists an index i, with $1 \leq i \leq h$, such that the y-coordinates of the points are strictly monotonically increasing from q_1 to q_i and strictly monotonically decreasing from q_i to q_h.

SKETCH OF PROOF: $UH(S)$ can be seen as a convex piecewise function.
The array of y-coordinates $y_1, \dots, y_h$, where y_i is the y-coordinate of q_i is *bitonic* (the values first increase, then decrease), i.e., there is only one (local maximum) index i such that y_i is larger than y_{i-1} and y_{i+1} (assume that $y_0 = y_{h+1} = -\infty$). If there were 2 or more local maxima, then $CH(S) = UH(S) + LH(S)$ would not be convex.



How to combine the two upper hulls

DEFINITION: The *upper common tangent* of $UH(S_1)$ and $UH(S_2)$ is the segment that is tangent to both $UH(S_1)$ and $UH(S_2)$ such that both both $UH(S_1)$ and $UH(S_2)$ are below the segment.



How to compute the upper common tangent

LEMMA: There exists a sequential algorithm that, given $UH(S_1)$ and $UH(S_2)$ as inputs, finds the upper common tangent of $UH(S_1)$ and $UH(S_2)$ in $O(\log n)$ time.

SKETCH OF PROOF:

Let a (resp. b) be the point of $UH(S_1)$ (resp., $UH(S_2)$) of maximum y-coordinate.

Let p_i be the first point of $UH(S_i)$.

The points in S of max y-coordinate is in $UH(S) \Rightarrow$ at least one point between a and b is in $UH(S)$.

Let τ be the upper common tangent of $UH(S_1)$ and $UH(S_2)$.

1. One endpoint of τ, say z_2, is between p_2 and b (p_2 and b are included). Assume we know z_2.
2. The other endpoint of τ, say z_1, is a point in $UH(S_1)$ between p_1 and a (p_1 and a are included)

- The algorithm finds the point of $UH(S_1)$ that minimizes the angular coefficient
- The array of all angular coefficients of $\tau(z)$, with z being a point of in $UH(S_1)$ between p and a, is bitonic (the values first decrease, then increase)
- The algorithm can find z_1 using binary search on the array of angular coefficients (this array does not have to be computed explicitly as the angular coefficient induced by two points can be computed directly)

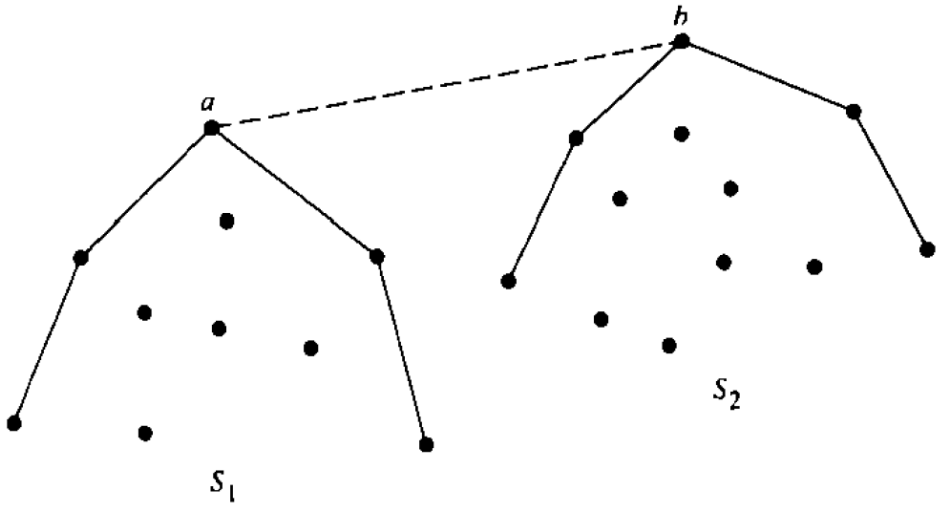
How to combine the two upper hulls

1. Let $UH(S_1) = q_1, \dots, q_s, UH(S_2) = q_1{}', \dots, q_t{}'$
2. Compute the upper common tangent, say it is (q_i, q'_j) /* $O(\log n)$ sequential time */
3. Then $UH(S) = q_1, \dots, q_i, q'_j, \dots, q'_t$ /* $O(1)$ time and $O(s+t) = O(n)$ work */

How to solve 3. in the time-processor framework. Code for processor with identifier k

- $UH(S)[k] = q_k$ if $k \leq i$ and $UH(S)[k] = q'{}_{k+j-i-1}$ if $i < k \leq i + t - j + 1$

The processor needs access to i, j and t (CREW PRAM)



The pseudocode for computing the upper hull

(It computes the upper hull of a set $S = \{p_1, \dots, p_n\}$ of $n = 2^k$ points sorted w.r.t. x-coordinate)

ALGORITHM Upper_Hull(S)

/* BASE CASE */

1. if n $\leq$ 4 **then** solve the problem directly using a sequential algorithm and **exit**

/* DIVIDE AND CONQUER */

2. **for** 1 $\leq$ i $\leq$ n **pardo if** 1 $\leq$ i $\leq$ n/2 **then** $S_1[i] \leftarrow p_i$ **else** $S_2[i-n/2] \leftarrow p_i$
3. **for** 1 $\leq$ i $\leq$ 2 **pardo** $UH(S_i) \leftarrow$ Upper_Hull(S_i)

/* COMBINE */

4. Deduce $UH(S)$ from $UH(S_1)$ and $UH(S_2)$ /* $O(\log n)$ time (compute u.c. tangent), $O(n)$ work */

- Time $T(n) = T(n/2) + O(\log n) = O(\log^2 n)$
- Work $W(n) = 2W(n/2) + O(n) = O(n \log n)$ (work optimal)
- CREW PRAM algorithm
  - No concurrent write
  - Concurrent reads when we deduce $UH(S)$

Check you understanding

Show how we can modify the algorithm Upper_Hull so as it still runs in time $O(\log^2 n)$ time and work $O(n \log n)$ on an EREW PRAM.

Check you understanding

DIAMETER OF POINTS ON A LINE

Consider a set of n points on the line whose coordinates are contained in an unsorted array X. Design an EREW PRAM algorithm that computes the diameter $D = \max_{1 \le i < j \le n} |X[i] - X[j]|$ of X in $O(\log n)$ time and $O(n)$ work.

Check you understanding

MAXIMUM SUBARRAY

Consider an array A of n numbers. The maximum subarray of A is a subarray $A[\ell, r]$ that maximizes $\sum_{i=\ell}^{r} A[i]$. Design an EREW PRAM algorithm that computes the maximum subarray of A in $O(\log n)$ time and $O(n)$ work.

Check you understanding

PARALLEL CLOSEST PAIR OF POINTS

Consider a set $p_1, \ldots, p_n$ of n points on the plane. Design a CREW PRAM algorithm that computes the distance $D = \min_{1 \le i < j \le n} |p_i - p_j|$ of the closest pair of points in $O(\log^2 n)$ time and $O(n \log n)$ work.